

《机器学习基础》



序列生成模型与循环神经网络

https://github.com/styluck/mech_learn

自然语言处理(natural language processing)

- ▶ 人工智能和语言学领域的一个分支，使计算机能够理解、解释和产生人类语言的内容。
- ▶ **自然语言处理模型** 一个重要的用法就是通过之前的信息来决策当前的问题。

(比如就像我们看电影，我们要根据电影之前的情节，才能理解现在的情节。)

例子1：有一朵云飘在（）

例子2：我从小生长在中国，我可以说一口流利的（）

自然语言处理(natural language processing)

►人工智能和语言学领域的一个分支，使计算机能够理解、解释和产生人类语言的内容。

- 1.语音识别：将语音转换为文本。
- 2.自然语言理解：理解自然语言的意图、情感和语境。
- 3.机器翻译：将一种语言翻译成另一种语言。
- 4.文本挖掘和文本分析：从文本中提取有用信息和洞察。
- 5.语言生成：生成自然语言响应或文本。

自然语言处理(natural language processing)

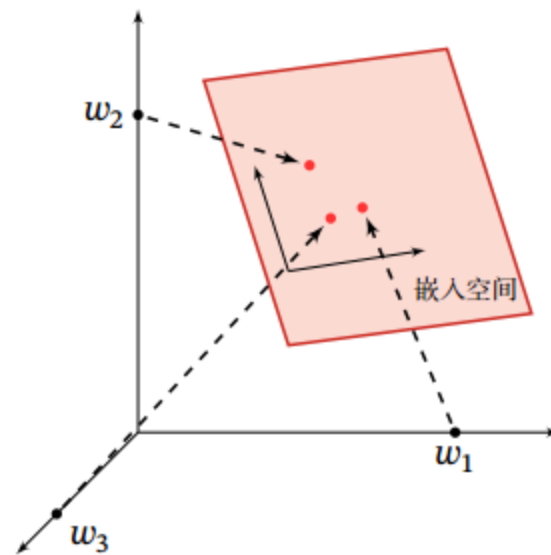
- ▶ 词袋模型 (Bag of Words, BoW) :把文本转换成数值向量的简单方法 (如一段话、一篇文章或一个文档) , 不考虑语法和词序, 但保留词汇的多重性 (即词汇出现的频率) 。
- ▶ One Hot编码 (One-Hot Encoding) :每个类别值都被转换成一个二进制向量, 除了表示该类别的一个位置是1之外, 其余位置都是0。这种编码方式因其表示的向量中只有一个元素是"热"的 (即1) , 其他位置都是"冷"的 (即0) , 因此得名One Hot编码。

什么是Embedding

- ▶使用One-hot 方法编码的向量会很高维也很稀疏。假设我们在做自然语言处理（NLP）中遇到了一个包含2000个词的字典，当使用One-hot编码时，每一个词会被一个包含2000个整数的向量来表示，其中1999个数字是0，如果字典再大一点，这种方法的计算效率会大打折扣。

什么是Embedding

- ▶ 把高维的局部表示空间映射到一个非常低维的分布式表示空间，例如 $[w_1, 0, 0]$ ，映射到一个更低维的稠密空间上。
- ▶ 在用深度学习做 NLP 任务时，通常我们会在输入层，即单词ID后接入一个 Embedding Layer，将整数转换为具有固定大小的向量。



什么是Embedding

▶ *deep learning is very deep*

- ▶ 使用嵌入层embedding 的第一步是通过索引对该句子进行编码，这里我们给每一个不同的单词分配一个索引，上面的句子就会变成这样：
- ▶ 1、2、3、4、1
- ▶ 我们要决定每一个索引需要分配多少个“潜在因子”，这大体上意味着我们想要多长的向量，通常使用的情况是长度分配为32和50。

什么是Embedding

- ▶ 假设每个索引指定6个潜在因子。嵌入矩阵则为

Indices	Latent Factors					
1	.32	.02	.48	.21	.56	.15
2	.65	.23	.41	.57	.03	.92
3	.45	.87	.89	.45	.12	.01
4	.65	.21	.25	.45	.78	.82

- ▶ 这可以使用嵌入矩阵而不是one-hot编码向量来保持每个向量更小。
- ▶ 单词“deep”用向量[.32, .02, .48, .21, .56, .15]来表达。
- ▶ 每一个单词都会被替换为用于查找嵌入矩阵中向量的索引。
- ▶ 大数据时也能有效计算。
- ▶ 可以探索在高维空间中哪些词语之间具有相似性



序列生成模型

序列数据的潜在规律

- ▶ 以自然语言为例，

面包上涂黄油

面包上涂袜子

- ▶ 后一个句子在人脑的语义整合时需要更多的处理时间，更不符合自然语言规则。
- ▶ 规则是概率：自然语言理解 $\rightarrow$ 一个句子中单词排序的可能性/合理性

序列概率模型

- ▶ 给定一个序列样本，其概率为

$$p(x_{1:T}) = p(x_1, x_2, \dots, x_T)$$

- ▶ 序列概率模型有两个基本问题：

- ▶ 学习问题：给定一组序列数据，估计这些数据背后的概率分布；
- ▶ 生成问题：从已知的序列分布中生成新的序列样本。

序列概率模型

- ▶ 给定一个序列样本，其概率为

$$p(x_{1:T}) = p(x_1, x_2, \dots, x_T)$$

- ▶ 序列数据有两个特点：

- ▶ (1) 样本的长度通常不固定；
- ▶ (2) 样本空间为非常大。

- ▶ 每个位置上的变量 x_t 的样本空间为一个给定的词表 $\mathcal{V}$ 。对于一个长度为 T 的序列，其样本空间为 $|\mathcal{V}|^T$ 。因此，我们很难用已知的概率模型来直接建模整个序列的概率。

序列概率模型

- ▶ 不能直接建模整个序列，所以要分解

$$\begin{aligned} p(x_{1:T}) &= p(x_1)p(x_2|x_1)p(x_3|x_{1:2}) \cdots p(x_t|x_{1:(t-1)}) \\ &= \prod_t p(x_t|x_{1:t-1}) \end{aligned}$$

- ▶ 因此，序列数据的概率密度估计问题可以转换为单变量的条件概率估计问题，即给定 $x_{1:t-1}$ 时 x_t 的条件概率 $p(x_t|x_{1:t-1})$ 。

序列概率模型

$$p(x_{1:T}) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdot \dots p(x_T|x_1, \dots x_{T-1})$$



$$p(x_{1:T}) = p(x_T) \cdot p(x_{T-1}|x_T) \cdot p(x_{T-2}|x_{T-1}, x_T) \cdot \dots p(x_1|x_2, \dots x_T)$$



物理上不一定可行

因果关系阻止反方向
模型的“错误”方向通常要复杂得多

序列概率模型

►学习：给定N个序列数据 $\{x_{1:T_n}^{(n)}\}_{n=1}^N$ ，序列概率模型需要学习一个模型 $p_\theta(x|x_{1:(t-1)})$ 来最大化整个数据集的对数似然函数。

$$\max_{\theta} \sum_{n=1}^N \sum_{t=1}^{T_n} \log p_\theta \left(x_t^{(n)} \middle| x_{1:(t-1)}^{(n)} \right)$$

自回归生成模型

- ▶ **生成**：一旦通过最大似然估计训练了模型 $p_{\theta}(x|x_{1:(t-1)})$ ，就可以通过时间顺序来生成一个完整的序列样本。
- ▶ 令 $\hat{x}_t$ 为在第 t 时根据分布 $p_{\theta}(x|\hat{x}_{1:(t-1)})$ 生成的词，

$$\hat{x}_t \sim p_{\theta}(x|\hat{x}_{1:(t-1)})$$

- ▶ 其中 $\hat{x}_{1:(t-1)} = \hat{x}_1, \dots, \hat{x}_{t-1}$ 为前面 $t - 1$ 步中生成的前缀序列。
- ▶ 每一步都需要将前面的输出作为当前步的输入，是一种**自回归** (autoregressive) 的方式。这一类模型也称为**自回归生成模型** (Autoregressive Generative Model)

自回归生成模型

▶ 自回归模型

$$p(x) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdot \dots p(x_T|x_1, \dots x_{T-1})$$



$$p(x_t|x_1, \dots x_{t-1}) = p(x_t|f(x_1, \dots x_{t-1}))$$

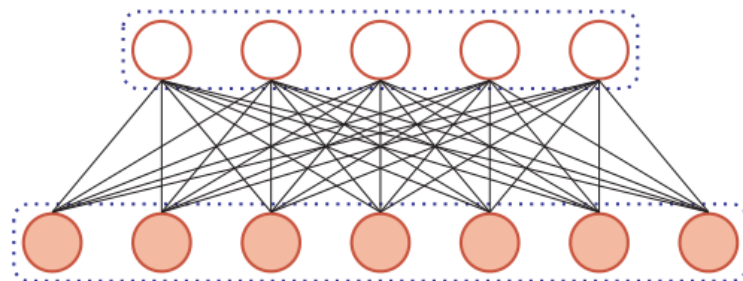
关于以前看到的数据的函数



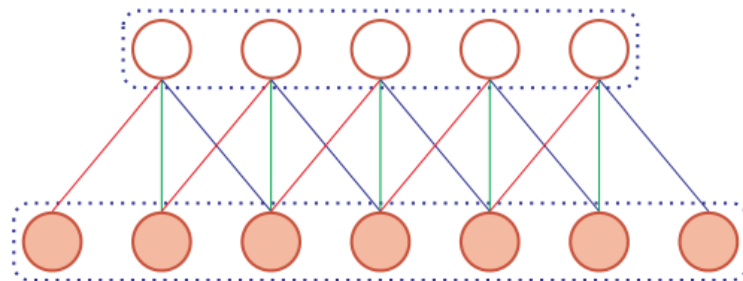
循环神经网络

前馈网络

- ▶ 连接存在层与层之间，每层的节点之间是无连接的。（无循环）
- ▶ 输入和输出的维数都是固定的，不能任意改变。无法处理变长的序列数据。



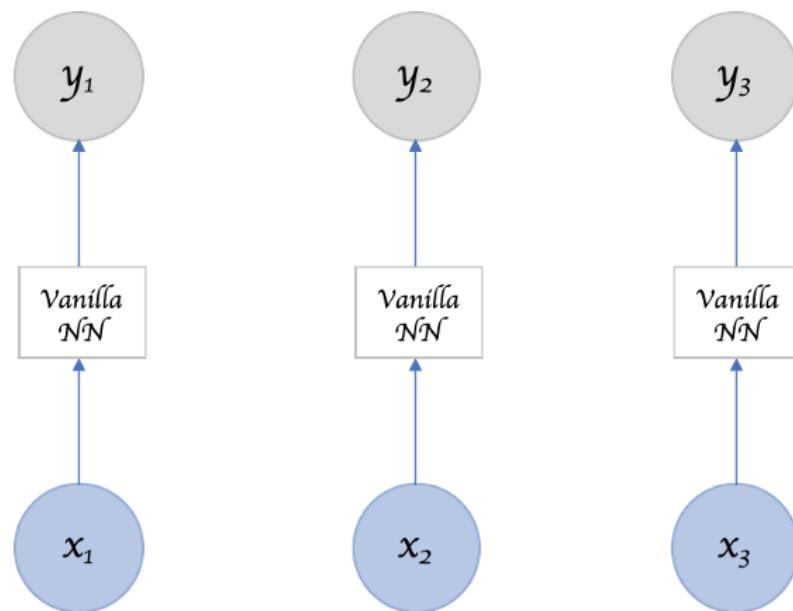
(a) 全连接层



(b) 卷积层

前馈网络

- 假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。

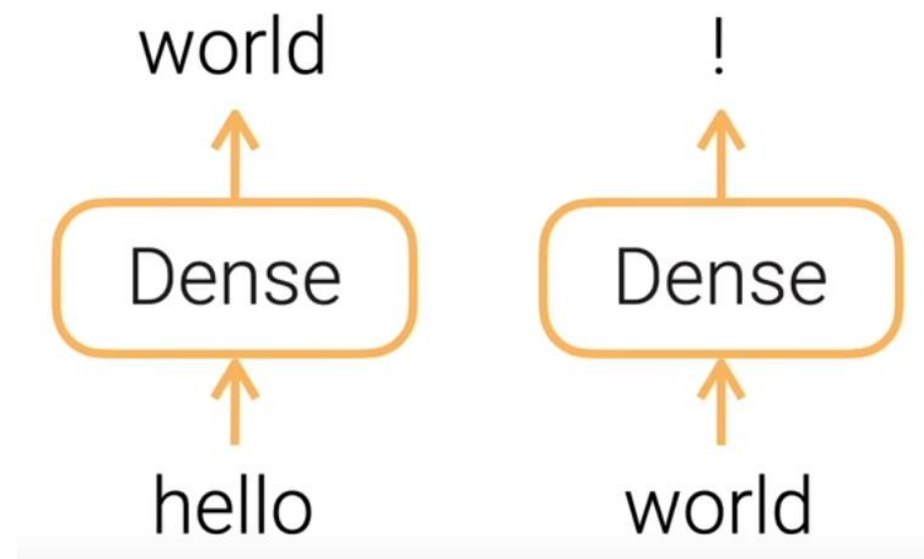


如果用前馈网络，会遇到什么问题？

▶如何预测下一个要输出的单词？

▶hello → world hello world → !

序列长度 $p_{\theta}(x_t | x_{1:t-1})$ 的输入长度会随着 t 增加。输入长度不同，MLP、CNN无法很好的处理时序数据。



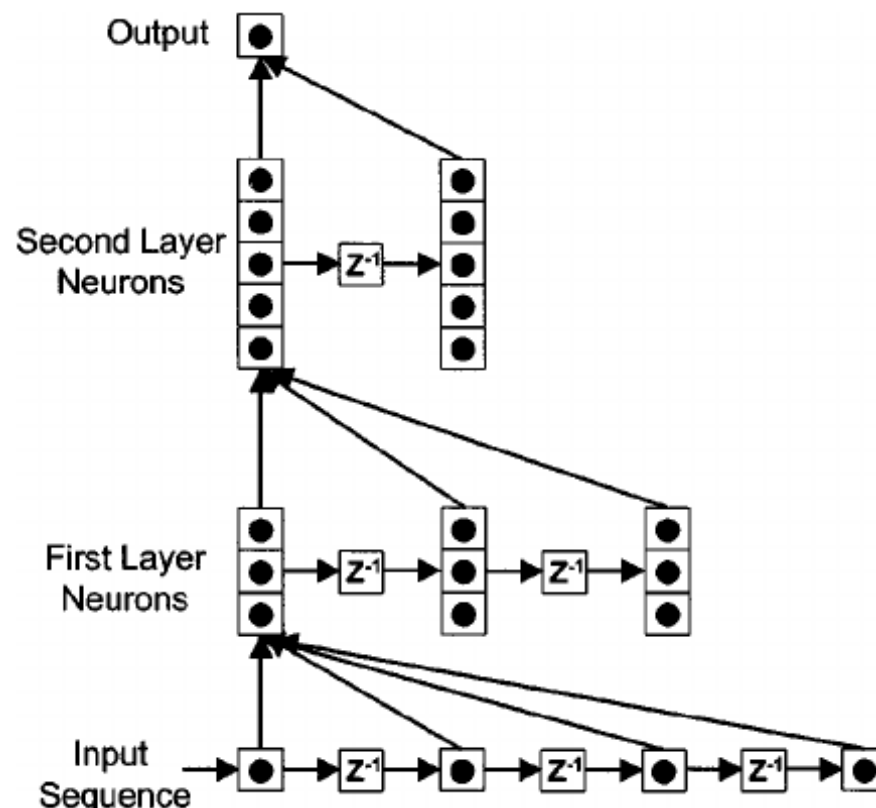
如何给网络增加记忆能力？

► 延时神经网络 (Time Delay Neural Network, TDNN)

- 建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）

$$\mathbf{h}_t^{(l)} = f(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l-1)}, \dots, \mathbf{h}_{t-K}^{(l-1)})$$

- 这样，前馈网络就具有了短期记忆的能力。



https://www.researchgate.net/publication/12314435_Neural_system_identification_model_of_human_sound_localization

自回归模型如何利用历史信息？

▶ 自回归模型 (Autoregressive Model, AR)

▶ 一类时间序列模型，用变量 y_t 自身的历史值预测当前值

$$y_t = w_0 + \sum_{k=1}^K w_k y_{t-k} + \epsilon_t$$

▶ $\epsilon_t \sim N(0, \sigma^2)$ 为第 t 个时刻的噪声， k 是窗口长度

▶ 有外部输入的非线性自回归模型 (Nonlinear Autoregressive with Exogenous Inputs Model, NARX)

$$y_t = f(x_t, x_{t-1}, \dots, x_{t-K_x}, y_{t-1}, y_{t-2}, \dots, y_{t-K_y})$$

▶ 其中 $f(\cdot)$ 可以由一个前馈神经网络表示。

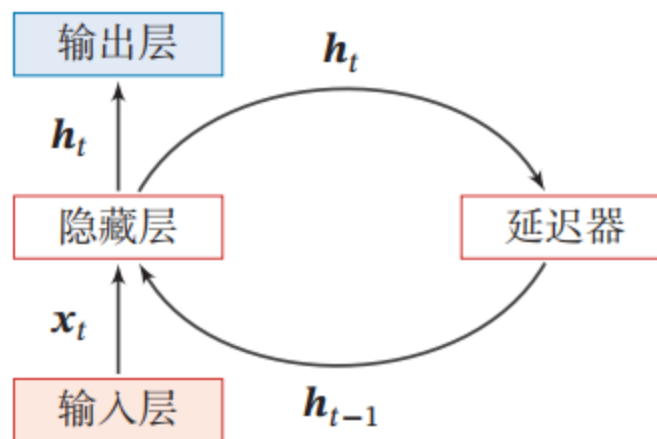
循环神经网络 (Recurrent Neural Network , RNN)

- ▶ 上述的TDNN和 AR / NARX 通常依赖一个固定长度的历史窗口。
为了处理更长、更灵活的历史信息，我们引入 RNN：
- ▶ 把历史信息压缩成固定长度状态： $h_t = \text{summary}(x_1, \dots, x_t)$.
- ▶ 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h),$$

$$y_t = W_{ho}h_t + b_o,$$

$$p_{\theta}(x_{t+1} | x_{1:t}) = \text{softmax}(y_t).$$



循环神经网络 (Recurrent Neural Network , RNN)

► RNN 如何生成文本？

输入起始符号；

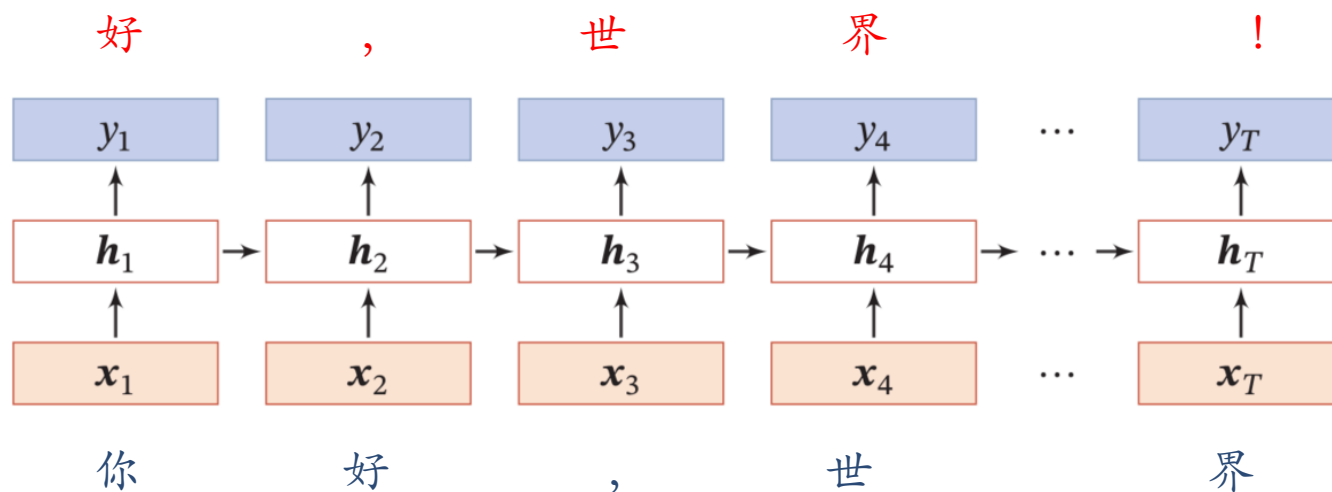
得到隐藏状态；

softmax 得到下一个词概率；

采样或取最大概率词；

把生成的词再输入模型；

重复直到生成结束符。



与 $\hat{x}_t \sim p_{\theta}(x|\hat{x}_{1:(t-1)})$ 类似！

循环神经网络的计算能力：通用近似定理

► 一个完全连接的循环网络是任何非线性动力系统的近似器。

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]: 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元, 它可以以任意的准确率去近似任何一个非线性动力系统

$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{x}_t), \quad (6.10)$$

$$\mathbf{y}_t = o(\mathbf{s}_t), \quad (6.11)$$

其中 $\mathbf{s}_t$ 为每个时刻的隐状态, $\mathbf{x}_t$ 是外部输入, $g(\cdot)$ 是可测的状态转换函数, $o(\cdot)$ 是连续输出函数, 并且对状态空间的紧致性没有限制.

循环神经网络的计算能力：图灵完备

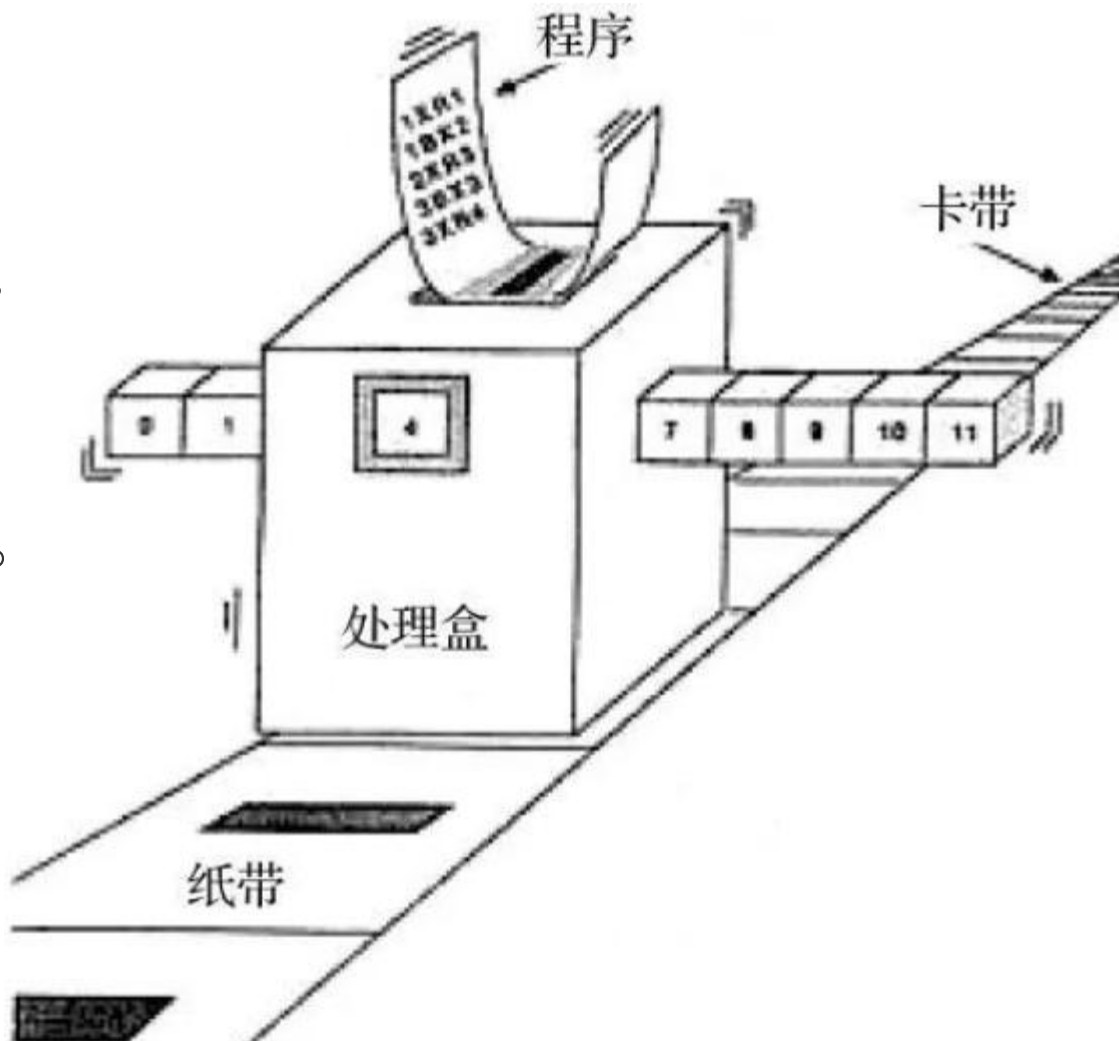
- ▶ 图灵完备 (Turing Completeness) 一个计算系统如果能够模拟图灵机的所有功能，那么它就是图灵完备的。这意味着该系统可以执行任何可计算的任务，无论多么复杂。
- ▶ 从功能上来说，图灵完备的系统具有无限的计算能力，只要有足够的时间和内存，它就可以解决任何可计算问题。

定理 6.2 – 图灵完备 [Siegelmann et al., 1991]: 所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

- ▶ 一个完全连接的循环神经网络可以近似解决所有的可计算问题。

图灵机

- ▶ 用纸笔进行数学运算的过程进行抽象，由一个虚拟的机器替代人类进行数学运算。
- ▶ 图灵机可以通过简单的读单元格、查指令集表、改变单元格状态、移动纸带这些非常简单、基本的操作来进行非常复杂的数学运算。
- ▶ 图灵机为确定哪些问题是可以计算的提供了一个标准。如果一个问题可以用图灵机解决，那么这个问题就是可计算的；否则，它是不可计算的。



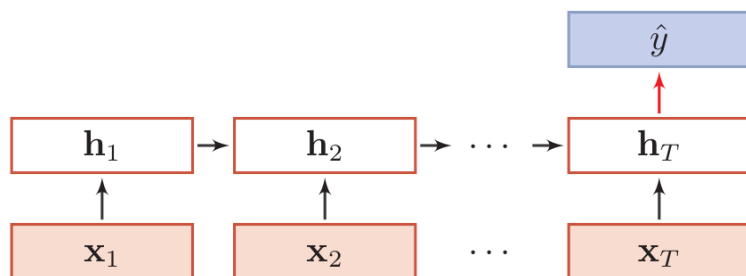


循环神经网络的应用

应用到机器学习

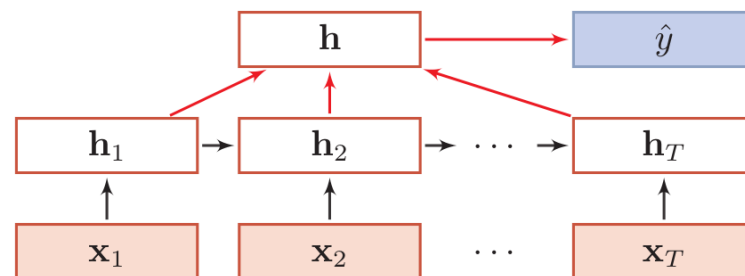
- ▶ 序列到类别
- ▶ 同步的序列到序列
- ▶ 异步的序列到序列

序列到类别



(a) 正常模式

$$\hat{y} = g(\mathbf{h}_T),$$

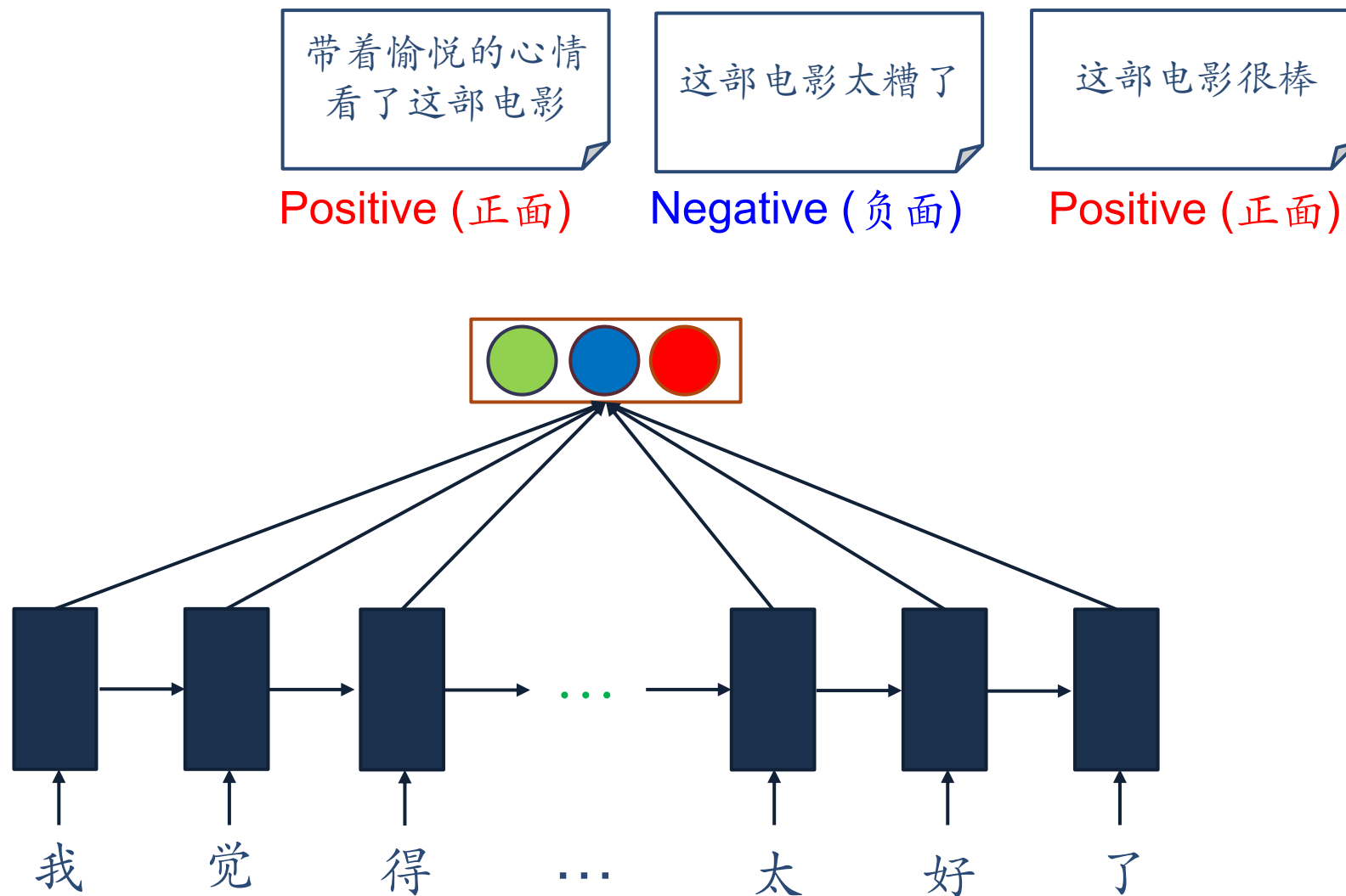


(b) 按时间进行平均采样模式

$$\hat{y} = g\left(\frac{1}{T} \sum_{t=1}^T \mathbf{h}_t\right).$$

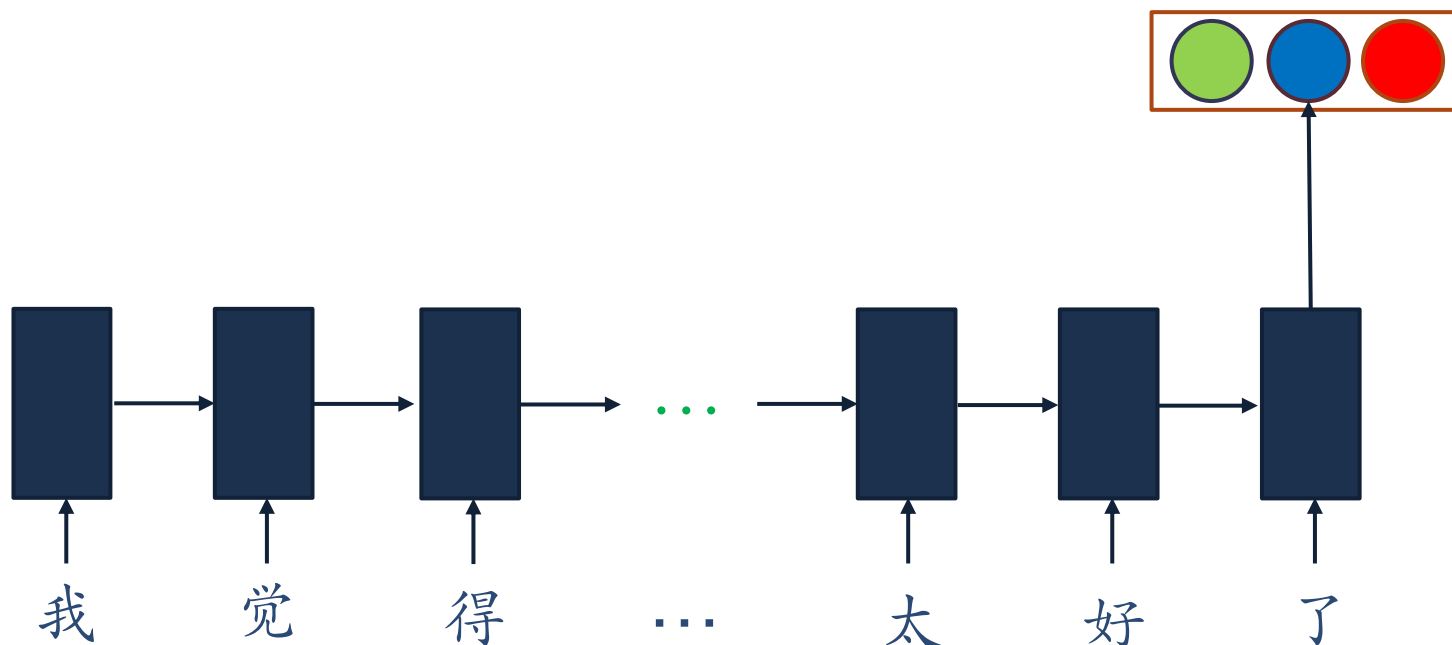
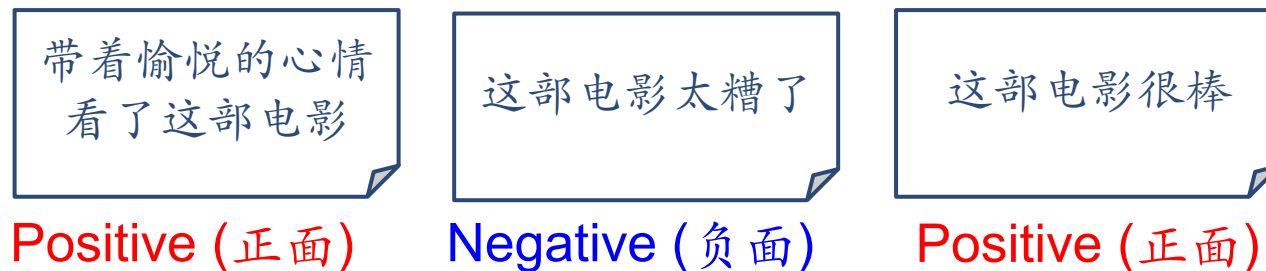
序列到类别

► 情感分类



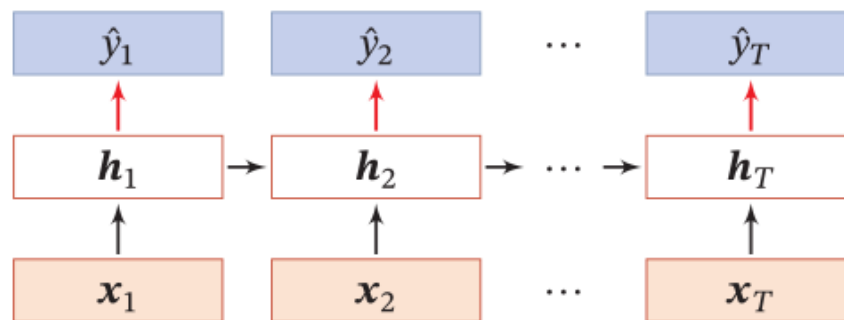
序列到类别

► 情感分类



同步的序列到序列模式

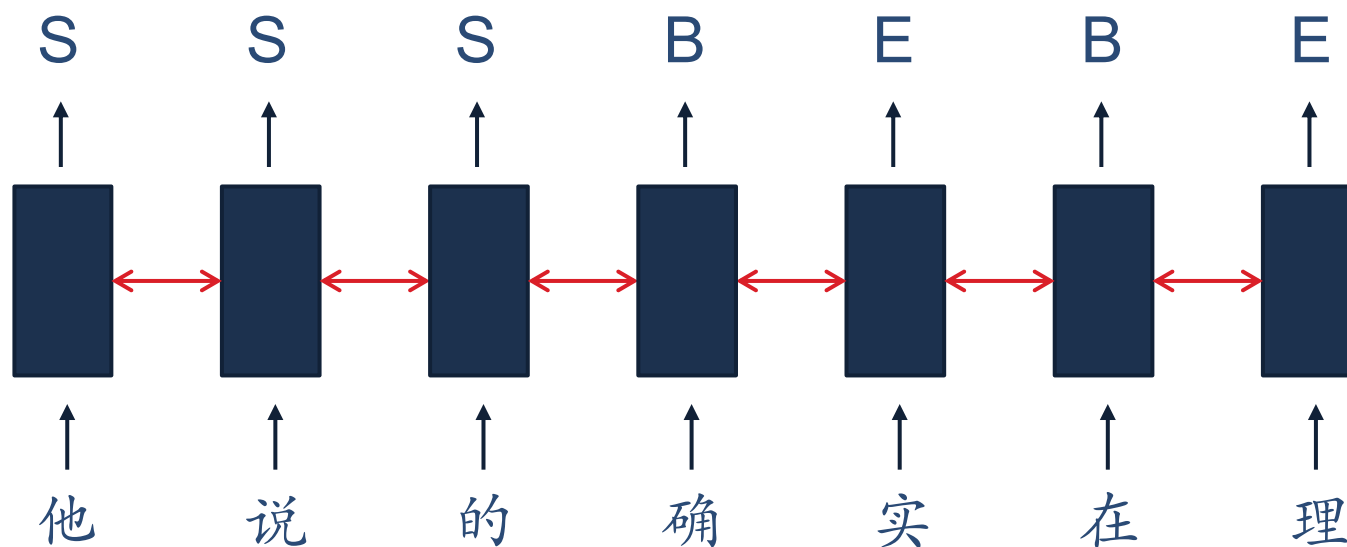
► 同步的序列到序列模式



$$\hat{y}_t = g(\mathbf{h}_t), \quad \forall t \in [1, T].$$

同步的序列到序列模式

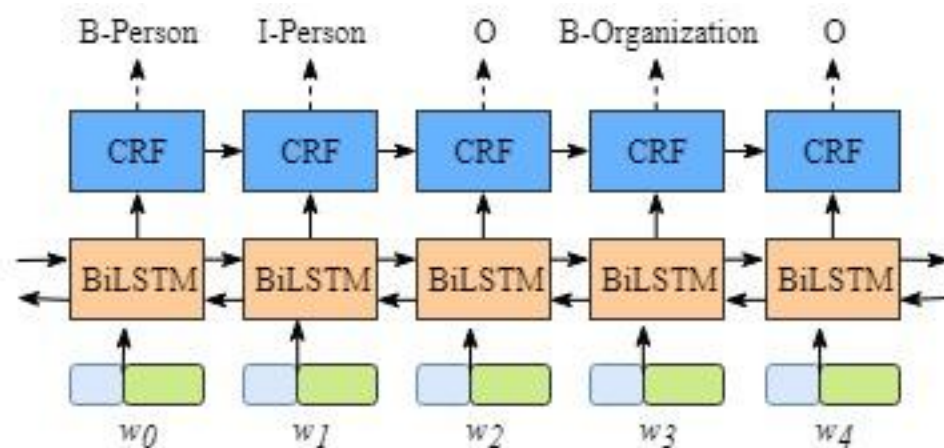
► 中文分词



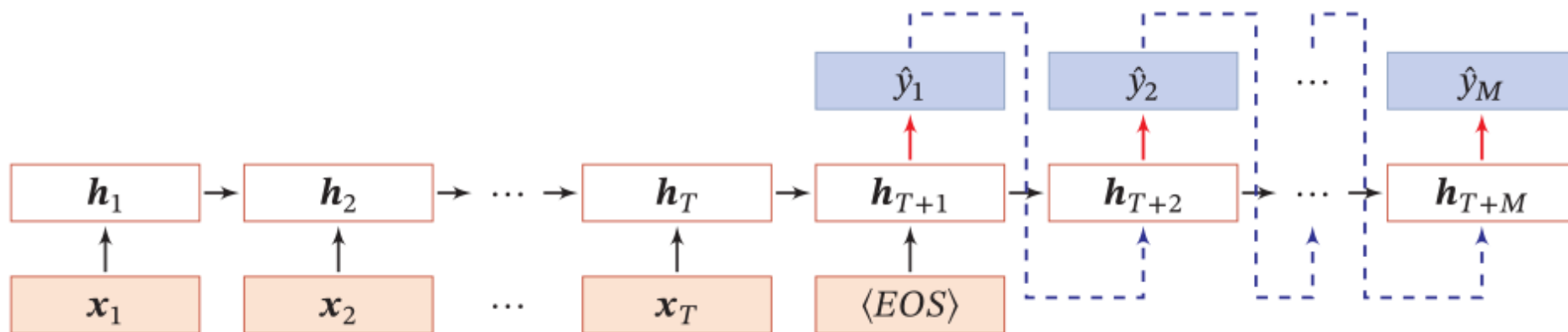
同步的序列到序列模式

- ▶ 信息抽取(Information Extraction, IE)
- ▶ 从无结构的文本中抽取结构化的信息，形成知识

小米创始人雷军表示，该公司2015年营收达到780亿元人民币，较2014年的743亿元人民币增长了5%。



异步的序列到序列模式



$$\mathbf{h}_t = f_1(\mathbf{h}_{t-1}, \mathbf{x}_t),$$

$$\forall t \in [1, T]$$

$$\mathbf{h}_{T+t} = f_2(\mathbf{h}_{T+t-1}, \hat{\mathbf{y}}_{t-1}),$$

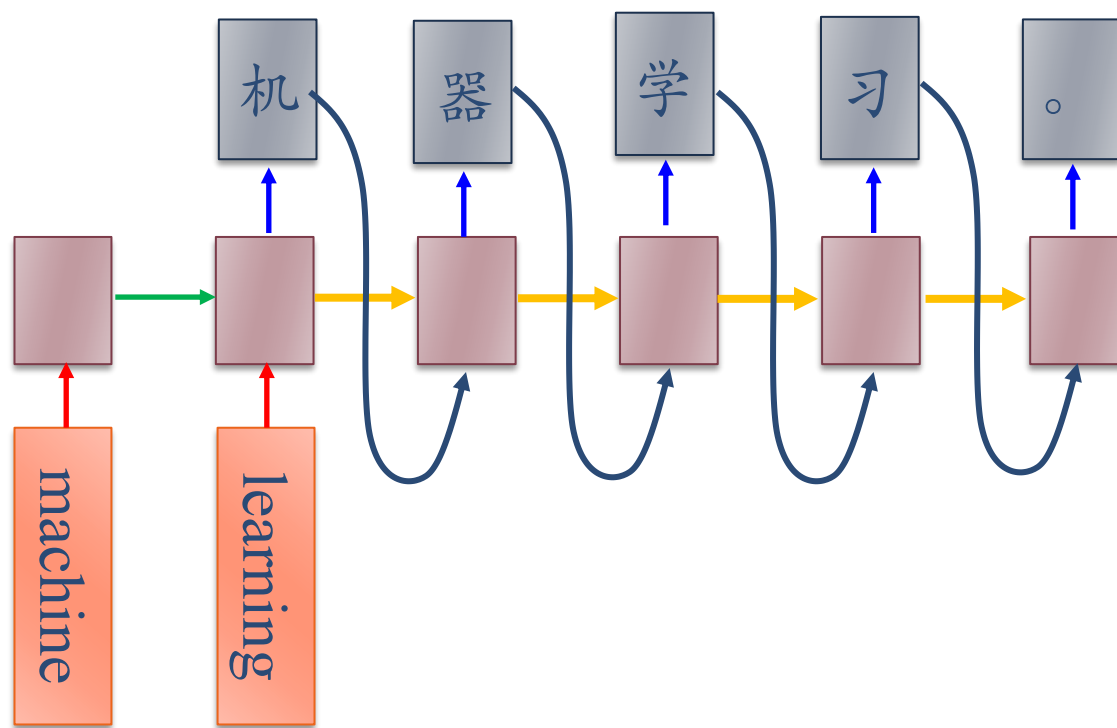
$$\forall t \in [1, M]$$

$$\hat{\mathbf{y}}_t = g(\mathbf{h}_{T+t}),$$

$$\forall t \in [1, M]$$

异步的序列到序列模式

► 机器翻译





参数学习

循环神经网络的参数学习

► 机器学习

► 给定一个训练样本 (x, y) ，其中

► $x = (x_1, \dots, x_T)$ 为长度是 T 的输入序列，

► $y = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

► 时刻 t 的瞬时损失函数为

$$\mathcal{L}_t = \mathcal{L}(\mathbf{y}_t, g(\mathbf{h}_t)),$$

► 总损失函数

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

► 梯度

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial \mathbf{U}}, \quad \longleftarrow \quad \frac{\partial \mathcal{L}_t}{\partial u_{ij}} = \sum_{k=1}^t \frac{\partial^+ \mathbf{z}_k}{\partial u_{ij}} \frac{\partial \mathcal{L}_t}{\partial \mathbf{z}_k},$$

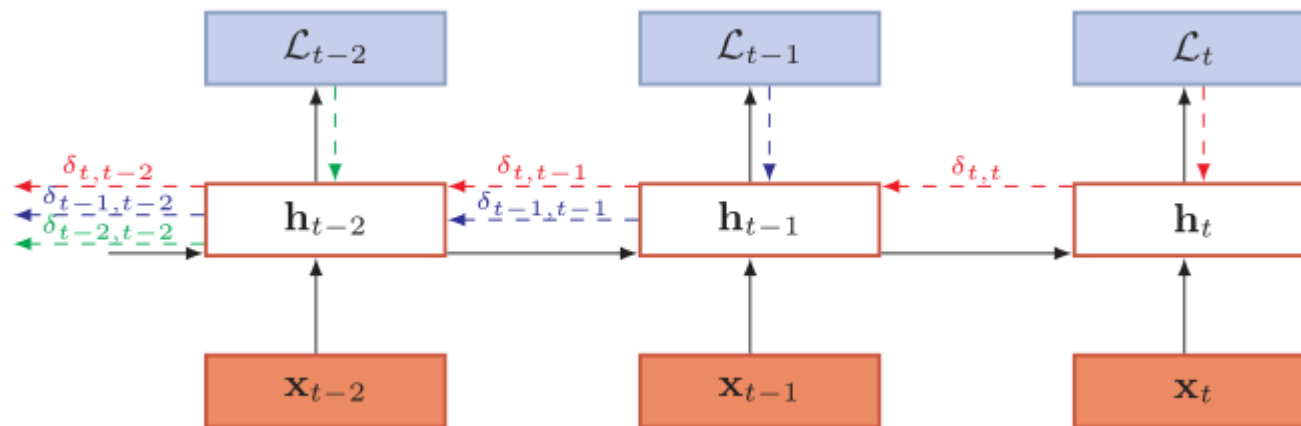
$$h_t = f(Uh_{t-1} + Wx_t + b),$$

$$\hat{y}_t = g(h_t),$$

$$\mathbf{z}_t = U\mathbf{h}_{t-1} + W\mathbf{x}_t + b$$

随时间反向传播算法

$$\mathbf{h}_{t+1} = f(\mathbf{z}_{t+1}) = f(U\mathbf{h}_t + W\mathbf{x}_{t+1} + \mathbf{b})$$



$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\text{diag}(f'(\mathbf{z}_\tau)) U^T \right) \delta_{t,t}$$

$\delta_{t,k}$ 为第 t 时刻的损失对第 k 步隐藏神经元的净输入 $\mathbf{z}_k$ 的导数

梯度消失/爆炸

► 梯度

► 其中

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{x}_k^T,$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k}.$$

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \underbrace{\left(\text{diag}(f'(\mathbf{z}_\tau)) U^T \right)}_{\lambda} \delta_{t,t}$$

由于梯度爆炸或消失问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题。

长程依赖问题

- ▶ 循环神经网络在时间维度上非常深!
 - ▶ 梯度消失或梯度爆炸
 - ▶ 很难建模长时间间隔 (Long Range) 的状态之间的依赖关系
- ▶ 如何改进?
 - ▶ 梯度爆炸问题
 - ▶ 权重衰减
 - ▶ 梯度截断
 - ▶ 梯度消失问题
 - ▶ 改进模型

梯度

▶ 反向传播的长程依赖问题

- ▶ 需要在内存中保留很多中间值
- ▶ 梯度消失或梯度爆炸

▶ 裁剪梯度以防止发散

$$\mathbf{g} \leftarrow \min \left(1, \frac{\theta}{\|\mathbf{g}\|} \right) \mathbf{g}$$

▶ 重新缩放到最大尺寸为 θ 的梯度

长程依赖问题

► 改进方法

► 循环边改为线性依赖关系

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta),$$

► 增加非线性

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta),$$

长程依赖问题

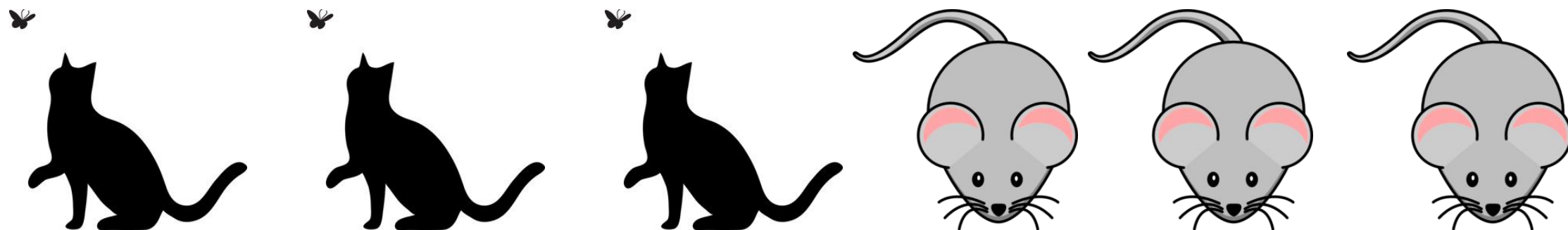
1. **长短期记忆网络 (LSTM)** : 通过引入门控机制, LSTM可以学习控制信息的流动, 从而更好地捕捉长期依赖关系。
2. **门控循环单元 (GRU)** : GRU是LSTM的一个变种, 它将LSTM中的遗忘门和输入门合并为一个更新门, 简化了模型结构。
3. ***注意力机制 (Attention Mechanism)** : 注意力机制允许模型在每个时间步动态地聚焦于序列中的不同部分, 从而更好地处理长期依赖。



长短期记忆神经网络 门控循环单元

在一个序列的注意力

► 并非所有元素都具有同等意义



► 只记住相关的元素

- 需要注意的机制（更新门）
- 需要忘记的机制（重置门）

RNN

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}),$$

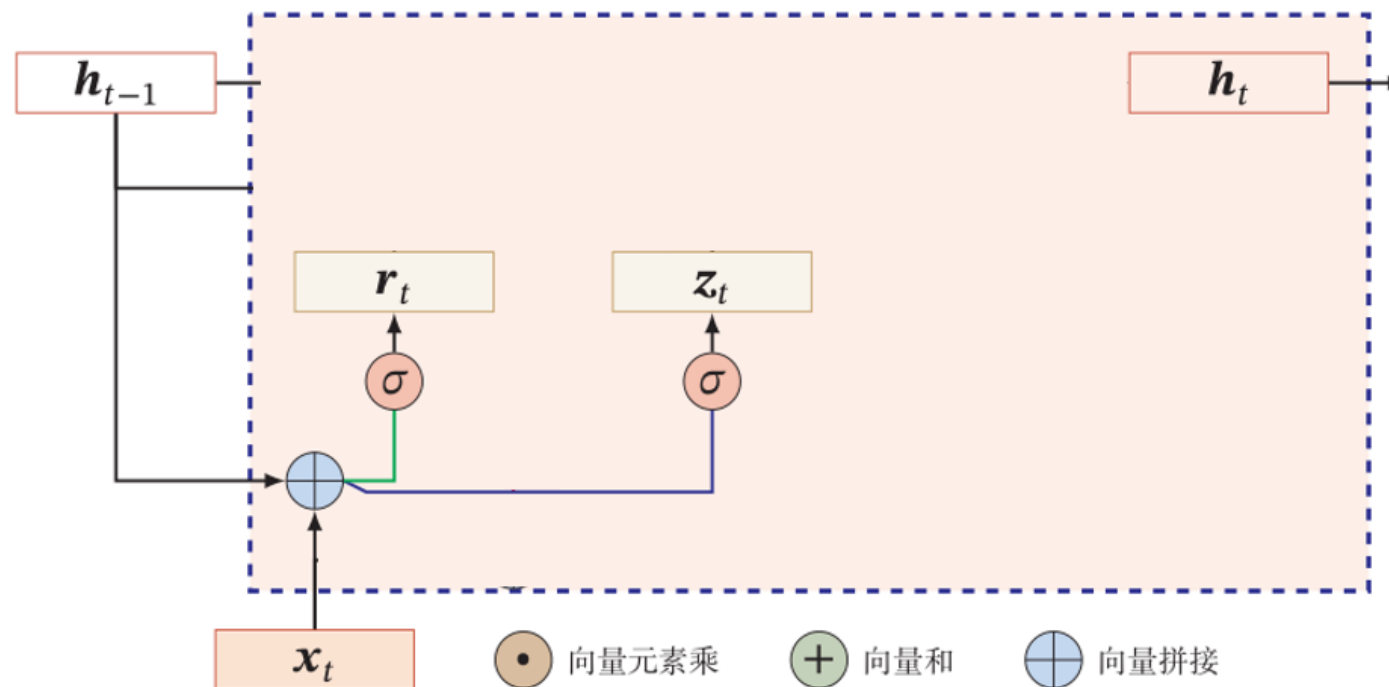
$$\mathbf{y}_t = \mathbf{V}\mathbf{h}_t,$$



门

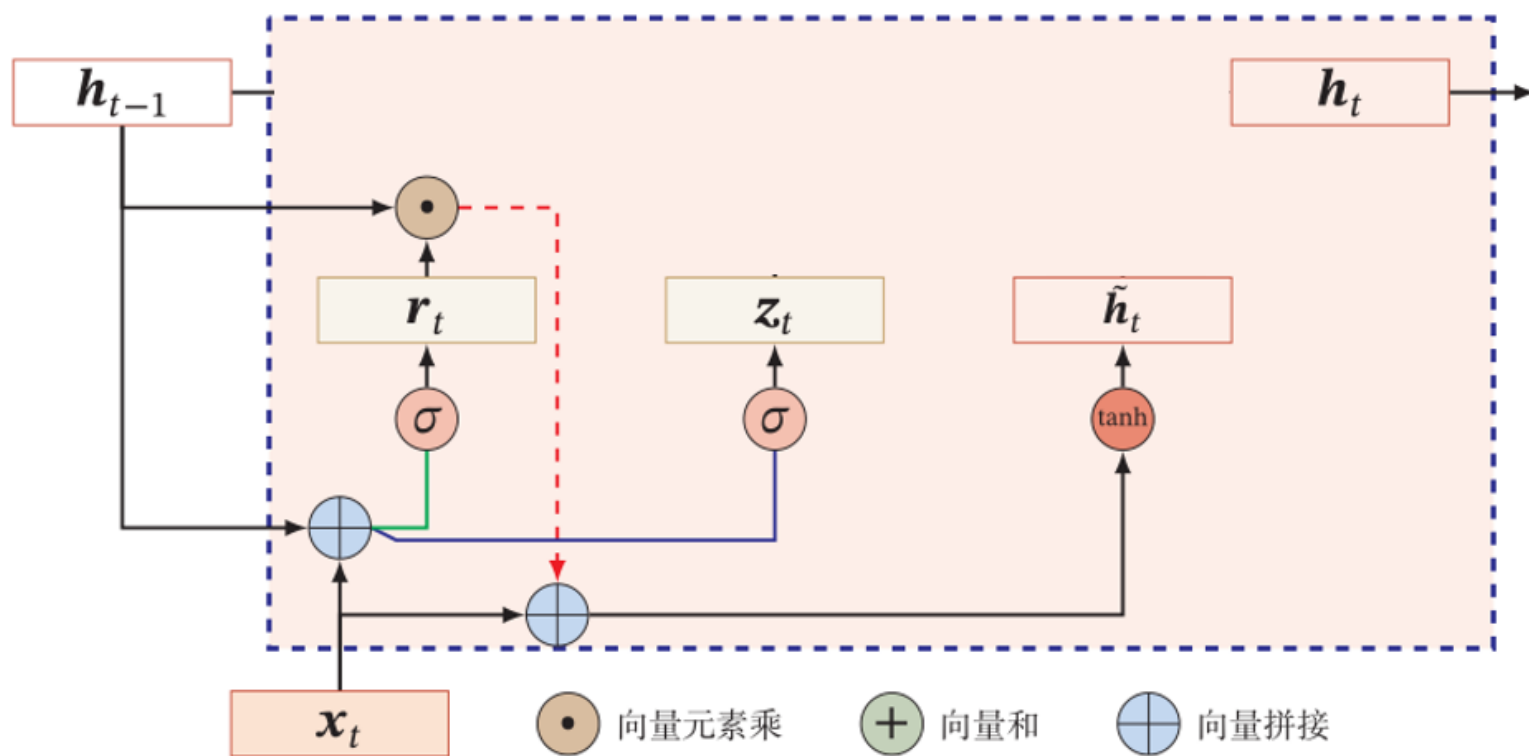
$$r_t = \sigma(x_t W_{xr} + h_{t-1} W_{hr} + b_r),$$

$$z_t = \sigma(x_t W_{xz} + h_{t-1} W_{hz} + b_z)$$



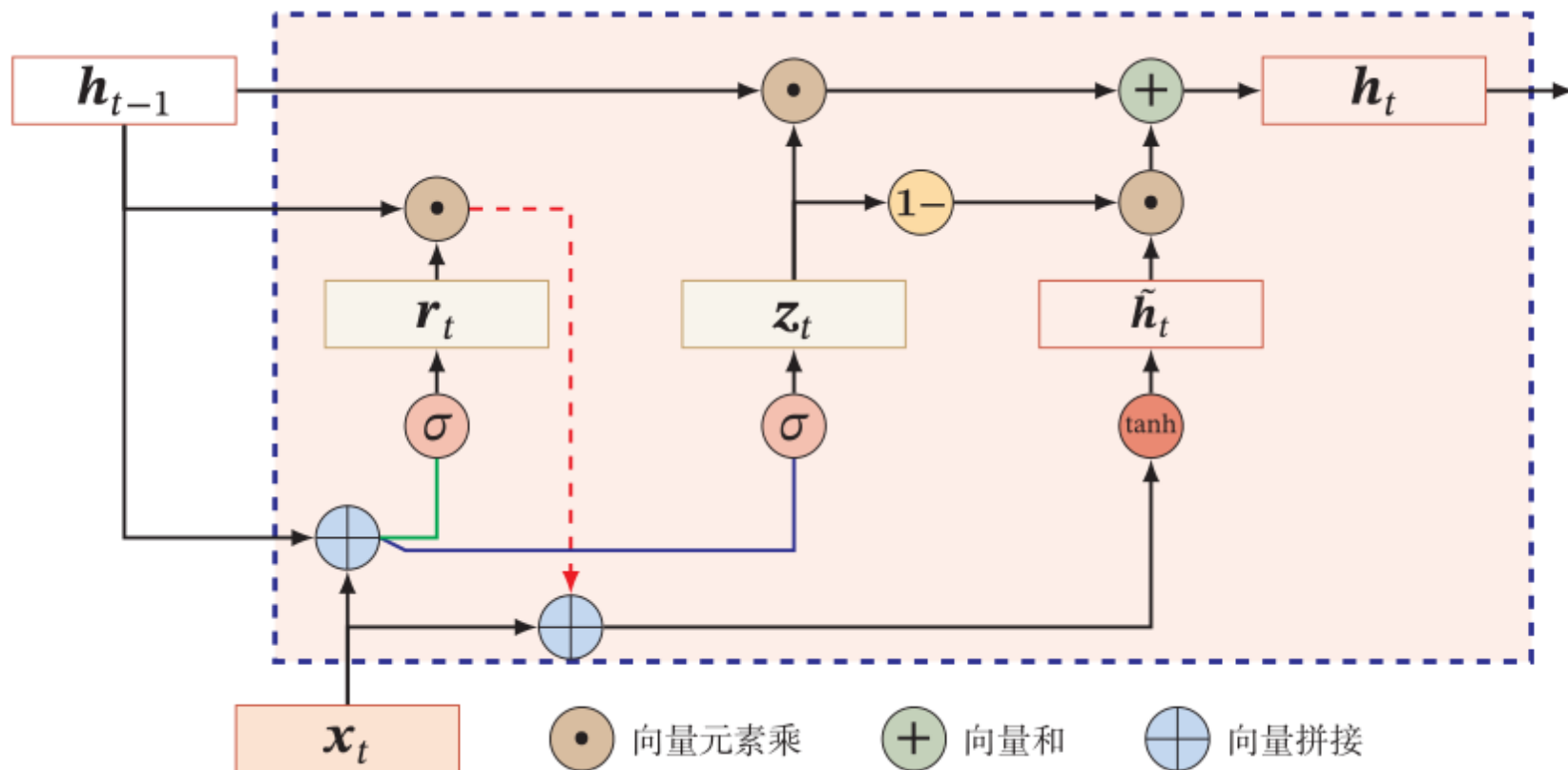
候选隐含状态

$$\tilde{h}_t = \tanh(x_t \mathbf{W}_{xh} + (r_t \odot h_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$

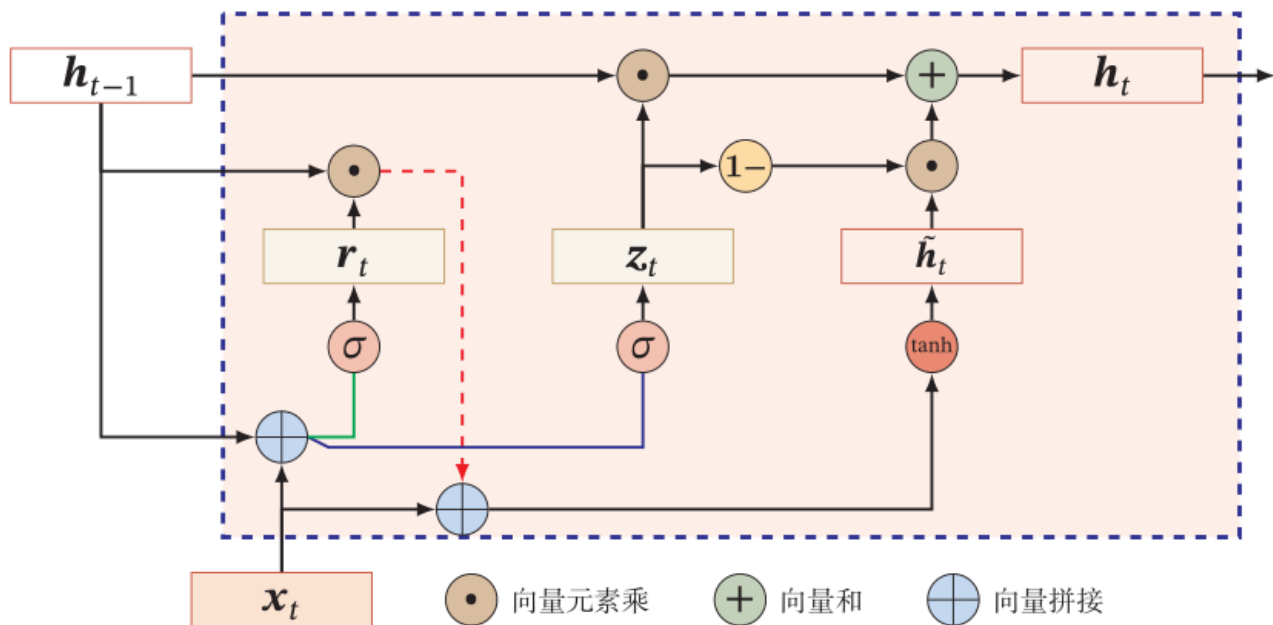


隐含状态

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$



门控循环单元 (GRU) 总结



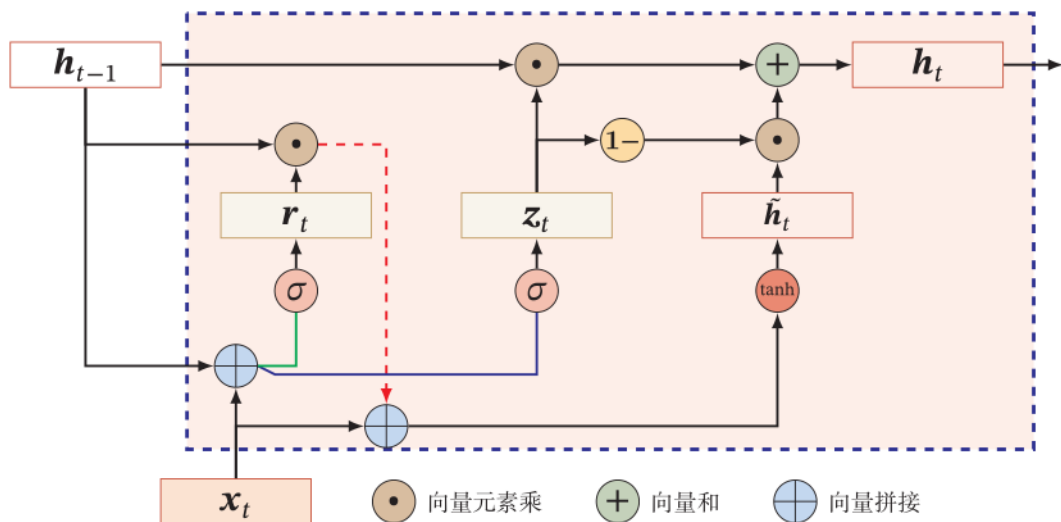
$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

门控循环单元 GRU



$$h_t = h_{t-1} + g(x_t, h_{t-1}; \theta),$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot g(x_t, h_{t-1}; \theta),$$

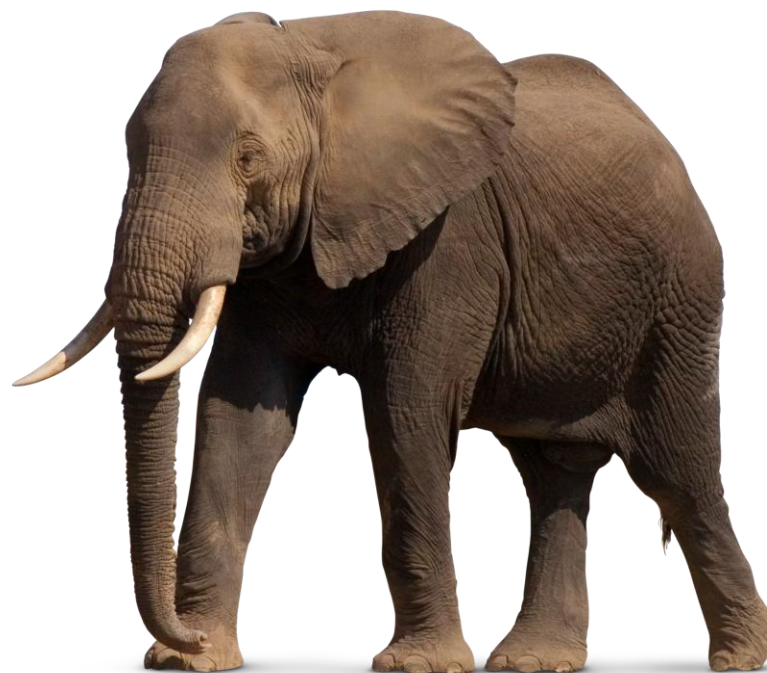
重置门 $\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r),$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}(\mathbf{r}_t \odot \mathbf{h}_{t-1}))$$

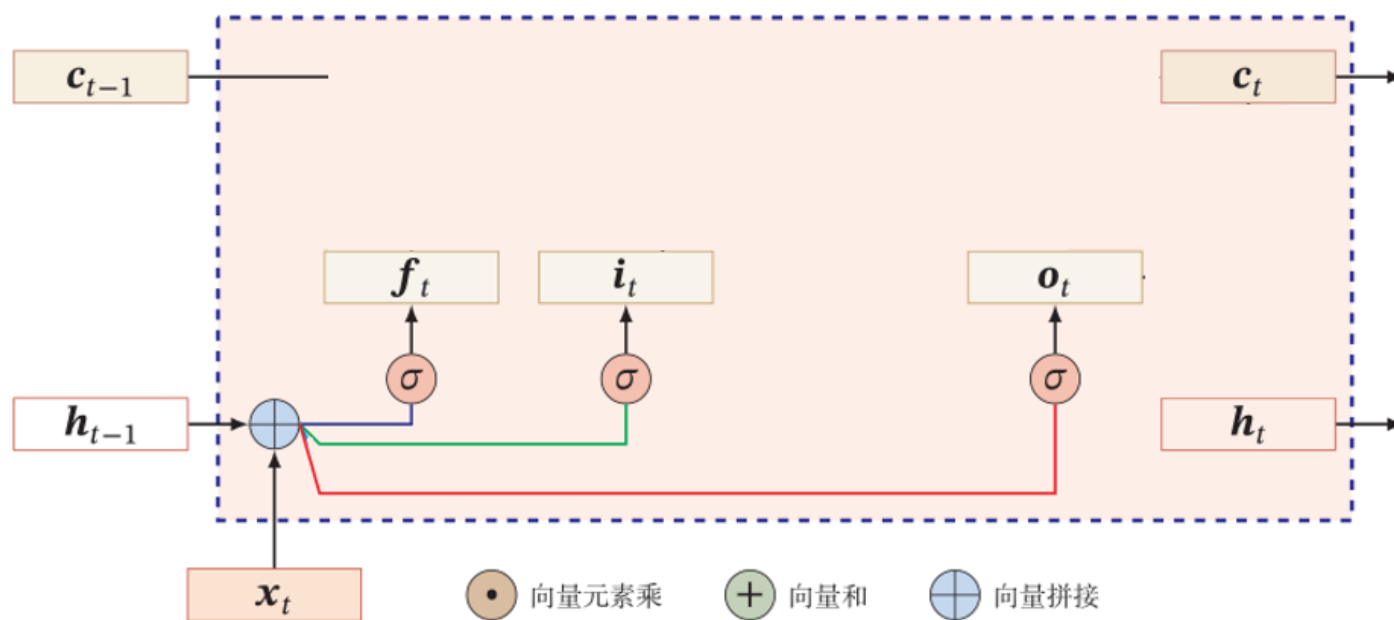
更新门 $\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z),$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t,$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)



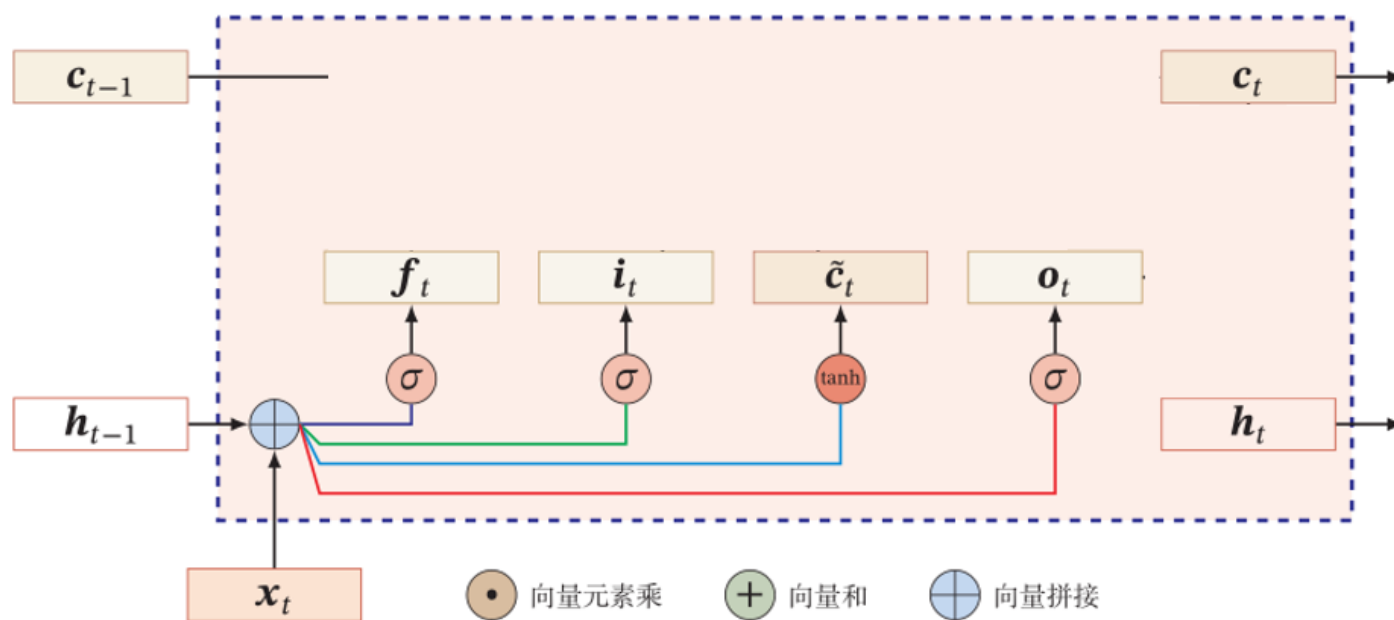
长短期记忆神经网络 (Long Short-Term Memory, LSTM)



- 遗忘门 f_t
- 控制上一个时刻
- 输入门 i_t
- 控制当前时刻
- 输出门 o_t
- 控制内部状态到外部状态

$$\begin{aligned}i_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), \\f_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), \\o_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o),\end{aligned}$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

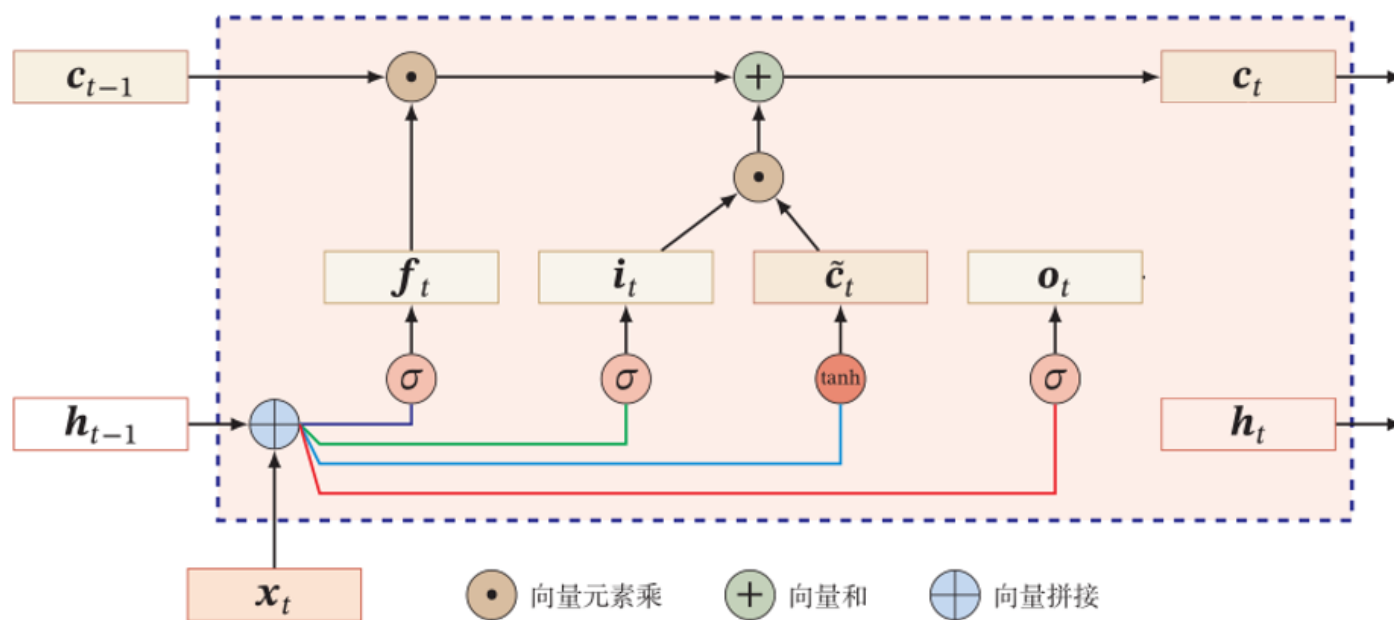


- 新的内部状态:
 $c_t \in \mathbb{R}^D$
专门进行线性的循环信息传递
- 多计算一个非线性的候选状态

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

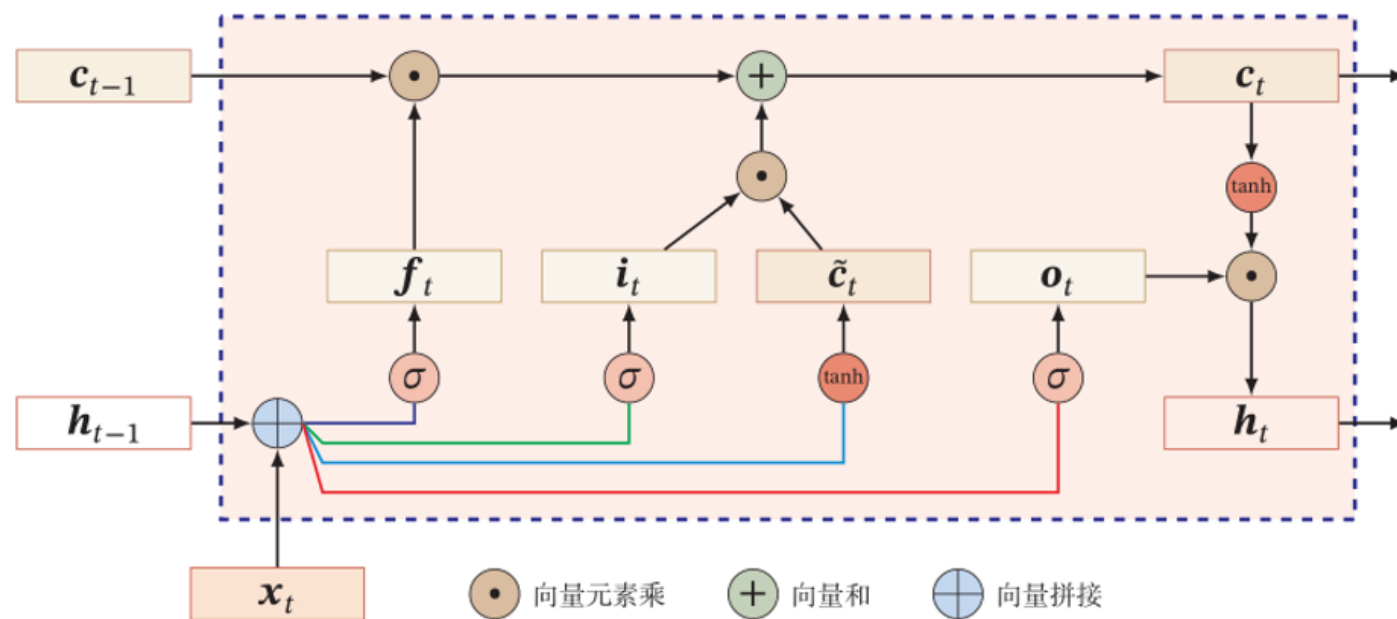


- 新的内部状态:
 $c_t \in \mathbb{R}^D$
专门进行线性的循环信息传递
- 多计算一个非线性的候选状态

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)



内部状态非线性地输出信息给外部的隐藏状态：

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

▶ 遗忘门 f_t

- ▶ 控制上一个时刻的内部状态 c_{t-1} 需要遗忘多少信息.

▶ 输入门 i_t

- ▶ 控制当前时刻的候选状态 $\tilde{c}_t$ 有多少信息需要保存.

▶ 输出门 o_t

- ▶ 控制当前时刻的内部状态 c_t 有多少信息需要输出给外部状态 h_t

▶ 内部状态

- ▶ LSTM网络引入一个新的内部状态专门进行线性的循环信息传递

▶ 长短期记忆

- ▶ 隐藏状态 h 的记忆比较短, 因此引入内部状态 c 获得更长的记忆力

LSTM的各种变体

▶ 没有遗忘门

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

▶ peephole连接

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + V_i \mathbf{c}_{t-1} + \mathbf{b}_i),$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + V_f \mathbf{c}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + V_o \mathbf{c}_t + \mathbf{b}_o),$$

▶ 耦合输入门和遗忘门

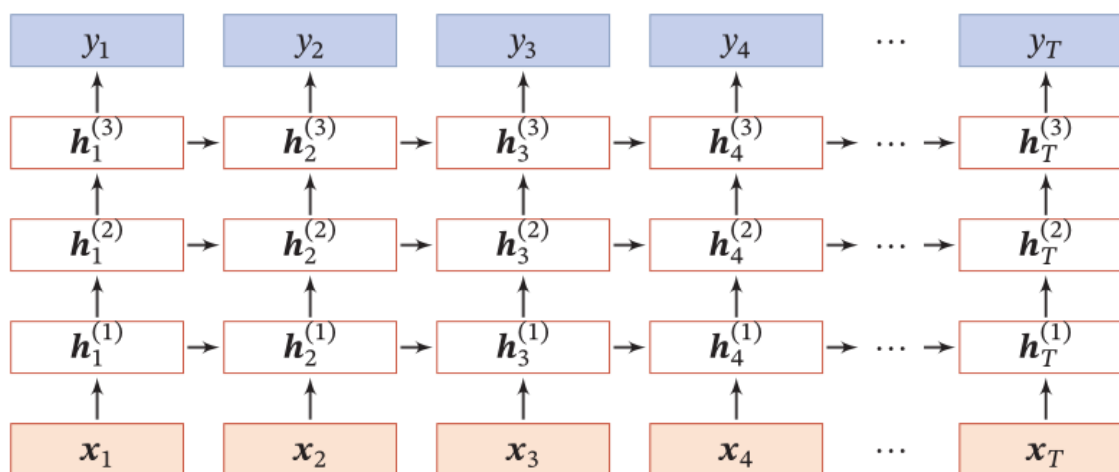
$$\mathbf{f}_t + \mathbf{i}_t = \mathbf{1}.$$

$$\mathbf{c}_t = (1 - \mathbf{i}_t) \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

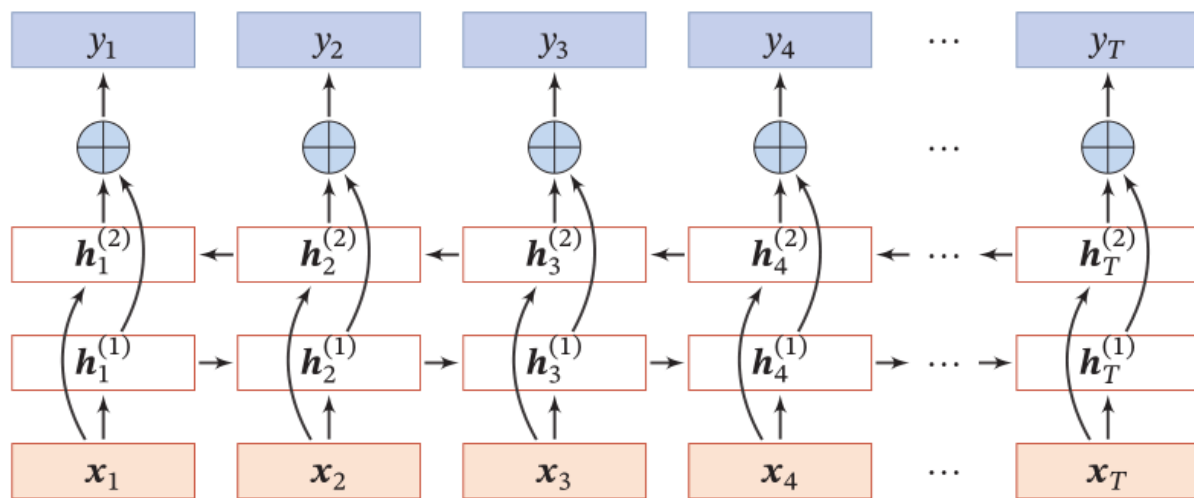


深层循环神经网络

堆叠循环神经网络



双向循环神经网络

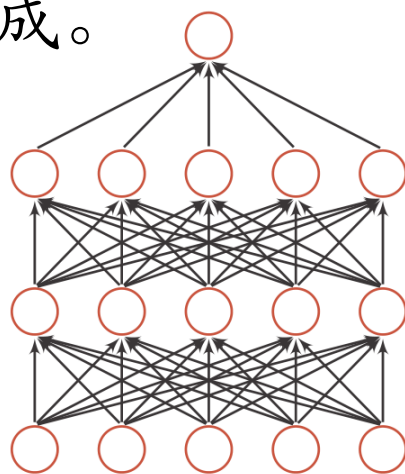


丢弃法 (Dropout Method)

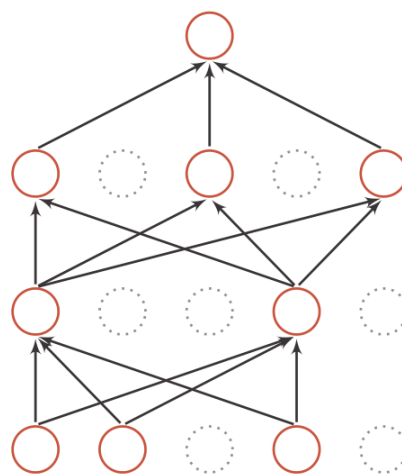
- ▶ 对于一个神经层 $y = f(Wx + b)$ ，引入一个丢弃函数 $d(\cdot)$ 使得 $y = f(Wd(x) + b)$ 。

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

- ▶ 其中 $m \in \{0,1\}^d$ 是丢弃掩码 (dropout mask)，通过以概率为 p 的贝努力分布随机生成。



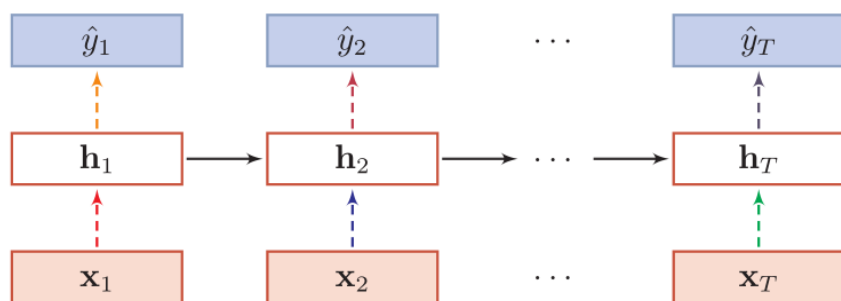
(a) 标准网络



(b) Dropout 后的网络

循环神经网络上的丢弃法

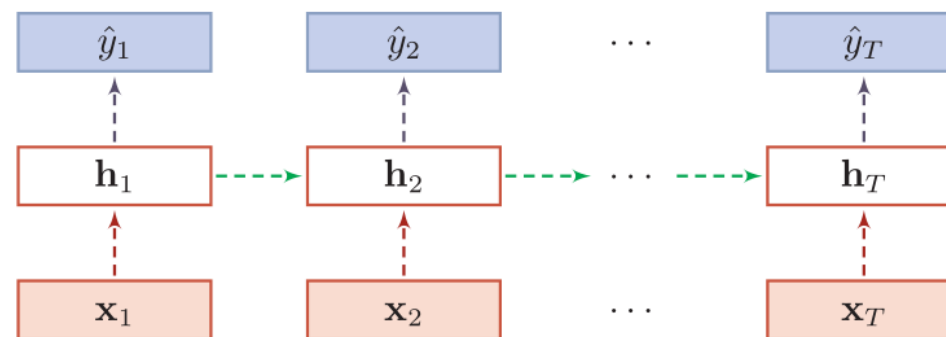
- ▶ 当在循环神经网络上应用丢弃法，不能直接对每个时刻的隐状态进行随机丢弃，这样会损害循环网络在时间维度上记忆能力。
- ▶ 可以对非时间维度的连接（即非循环连接）进行随机丢失：



虚线边表示进行随机丢弃，不同的颜色表示不同的丢弃掩码。

变分Dropout

- ▶ 根据贝叶斯学习的解释，丢弃法是一种对参数 θ 的采样。
- ▶ 每次采样的参数需要在每个时刻保持不变。因此，需要对参数矩阵的每个元素进行随机丢弃，并在所有时刻都使用相同的丢弃掩码。



相同颜色表示使用相同的丢弃掩码